Rapport complet d'implémentation, d'entraînement et d'audit `eLLM_v0`

Dossier de traçabilité technique

Version livrée : 0.2.0 – 21 octobre 2026

Résumé

Ce rapport décrit de bout en bout l'artefact effectivement implémenté sous le nom `eLLM_v0`, depuis les données et le découpage jusqu'à l'entraînement, aux empreintes multi-résolution, à la génération, à la recherche, à la mémoire facultative, à l'API HTTP, à l'interface Web, aux tests et aux archives. Les conclusions s'appuient sur les logs et résultats JSON fournis dans le paquet d'audit. L'implémentation est un modèle de langage probabiliste à n-grammes interpolés d'ordre variable, associé à un observateur d'empreintes explicites. Ce n'est pas un Transformer et ce n'est pas un modèle neuronal profond. Cette caractérisation est descriptive de l'artefact présent, et non une prédiction sur ce qu'une future version pourrait atteindre. Les tests établissent les propriétés qui ont été effectivement mesurées ; ils ne sont pas présentés comme preuve de compétence sémantique générale ni comme preuve empirique universelle de la théorie des empreintes.

Table des matières

1 Description **2**
- 1.1 Objet et périmètre . . . 2
- 1.2 Périmètre testé et hors périmètre . . . 2
- 1.3 Sources et identité des artefacts . . . 2

2 Architecture **2**
- 2.1 Modules et responsabilités . . . 2
- 2.2 Principe de séparation . . . 3

3 Pipeline complet **3**

4 Commandes d'exécution **4**
- 4.1 Prérequis et démarrage . . . 4
- 4.2 Réentraînement et tests . . . 4
- 4.3 Commandes d'interface et API . . . 5

5 Résultats **5**
- 5.1 Entraînement principal . . . 5
- 5.2 Répétitions de partition . . . 5
- 5.3 Sensibilité au prompt et choix de l'ordre . . . 6
- 5.4 Invariants des empreintes . . . 6
- 5.5 Serveur, CLI et interface . . . 6
- 5.6 Test de portabilité après extraction propre . . . 7

6 Interprétations des résultats **7**

7 Conséquences théoriques, conceptuelles, opérationnelles **7**
7.1 Théoriques 7
7.2 Conceptuelles 7
7.3 Opérationnelles 7

8 Conclusions **8**

9 Statut du modèle **8**

1 Description

1.1 Objet et périmètre

`eLLM_v0` vise à relier un prédicteur de prochain token à une couche observationnelle inspirée du socle `DDD_Base` : sondes, empreintes, signatures discrètes, classes d'indiscernabilité, fibres, quotients, seuils, fermeture sur un univers fini, capacité et dimension effectives, graphe relationnel et diagnostics de transport/factorisation. L'artefact inclut une CLI, un serveur local standard-library, une interface de chat responsive, des outils de recherche et d'imagination internes, ainsi qu'un journal de preuves.

1.2 Périmètre testé et hors périmètre

Le corpus livré comprend 80 paragraphes français. Le modèle de distribution est un interpolateur n-gramme appris à partir de comptages. La mémoire est optionnelle et locale. Le navigateur Chromium a testé l'interface DOM avec appels réseau simulés ; les routes réelles ont été testées indépendamment contre le serveur Python et le checkpoint livré. Les performances sur corpus externes, la factualité, la sécurité de contenu, l'évaluation humaine, la charge multi-utilisateur, l'accessibilité formelle et la compatibilité exhaustive avec tous les systèmes/navigateurs ne sont pas revendiquées comme testées.

1.3 Sources et identité des artefacts

Les références théoriques livrées sont `source_th_emp_llm_0.tex` et `source_emp_llm_0.json`, toutes deux version 0.2. Le checkpoint de livraison est `Dmodel_eLLM_v0/checkpoints/model.json` ; son SHA-256 est `3e42677b469937ba688022c30d968752c412c09eaabe3d826d2712f6e84590dc`. Le corpus a pour SHA-256 `0044c56325501e61a210a2b1571d13bd68565664f22dd64570fb3a173b69bc6f`. Les empreintes détaillées de chaque source sont enregistrées dans `prouves_eLLM_v0.json` et le manifeste global.

2 Architecture

2.1 Modules et responsabilités

Module	Responsabilité livrée
`tokenizer.py`	Tokenisation Unicode déterministe, vocabulaire, identifiants spéciaux et traitement des tokens hors vocabulaire.
`model.py`	Comptages d'ordre 0 à m, lissage interpolé, distribution de prochain token, échantillonnage, génération et sérialisation atomique JSON.
`train.py`	Chargement du corpus, séparation déterministe par document, entraînement et métriques.

`footprint.py`	Familles emboîtées de sondes suffixes, empreintes continues et signatures exactes, partitions, fibres, seuils, donnée de bord, mesure des seuils, fermeture, capacité/dimension effectives et maillage relationnel.
`diagnostics.py`	Proxies explicitement nommés pour les catégories de déficit ; ils ne sont pas des mesures absolues d'intelligence.
`transport.py`	Test de commutation restriction/transport, rapport de factorisation et descente d'opérations sur un univers fini.
`memory.py`	Mémoire JSON locale optionnelle, écriture, récupération heuristique et effacement.
`search.py`	Recherche déterministe par faisceau et trajectoires imaginées par échantillonnage à graines.
`engine.py`	Orchestration chat, préparation du prompt, trace observationnelle, exploration, diagnostic et imagination.
`server.py`	API HTTP locale, gestion d'état/sessions, validation de paramètres, journalisation et routes d'interface.
`web/`	Interface originale de chat local, responsive, avec conversations, traces, actions explorer/imaginer/diagnostiquer, métriques, thème et gestion mémoire.

2.2 Principe de séparation

Le token, l'état de calcul, la signature d'empreinte, la classe observationnelle, la signification supposée et la sortie textuelle ne sont pas identifiés. Une activation ou une probabilité est une observable numérique ; la sémantique n'est pas certifiée par la seule présence de cette observable. La recherche et l'imagination traversent les continuations ayant une probabilité apprise, sans réponse cible codée en dur.

3 Pipeline complet

1. **Corpus** : lire les 80 paragraphes UTF-8 ; conserver son empreinte cryptographique.
2. **Découpage** : séparer les documents avec graine déterministe en 64 entraînement, 8 validation, 8 test ; comparer les hashes de documents pour vérifier la disjonction.
3. **Tokenisation** : construire le vocabulaire uniquement à partir du train ; convertir les tokens en identifiants stables et ajouter EOS aux cibles.
4. **Apprentissage** : accumuler des tables de comptage pour les contextes de longueur 0 à 5 ; calculer les probabilités par interpolation lissée.
5. **Choix de configuration** : comparer 25 couples d'hyperparamètres sur la validation. La politique de livraison fixe l'ordre maximum à 5 pour conserver une sensibilité minimale au prompt, puis choisit le lissage par NLL validation. Une ablation distincte a observé une meilleure NLL de validation à ordre 2, mais ce contexte donnait une seule distribution identique sur les 8 prompts diagnostiques ; le contexte 5 est donc un compromis explicite, pas l'optimum universel de toute métrique.
6. **Évaluation réservée** : après gel de la configuration, évaluer le test de 8 documents et comparer le baseline unigramme sur les mêmes exemples.
7. **Observation** : tokeniser le contexte réellement reçu par le prédicteur ; pour chaque résolution k, calculer la famille de suffixes et la signature exacte, puis la distribution prédictive continue associée.
8. **Analyse structurelle** : construire partitions/fibres/quotients, seuils, mesure empirique, fermeture sur les sondes disponibles, dimension effective et relations de maillage sur des domaines

finis annoncés.

9. **Dialogue** : ajouter un prompt formaté et l'historique pertinent ; calculer la distribution, échantillonner les tokens jusqu'à EOS ou jusqu'au nombre demandé ; joindre une trace des niveaux observationnels.
10. **Outils internes** : Explorer lance un beam search reproductible ; Imaginer produit des branches échantillonnées avec graines explicites ; Diagnostics renvoie plusieurs proxies distincts ; Transport teste factorisation et commutation sur les données fournies.
11. **Interface/serveur** : le navigateur utilise les ressources locales. Le serveur HTTP local expose status, metrics, memory, chat, search, imagine, diagnostics, transport-check, reset et memory/clear.
12. **Audit** : tests unitaires/HTTP, Chromium, smoke sur serveur réel, réentraînement reproductible, invariants structurels, journaux d'incidents, savepoint, SHA-256 et archives contrôlées par CRC.

4 Commandes d'exécution

4.1 Prérequis et démarrage

Le runtime livré dépend de Python 3.10+ et de la bibliothèque standard ; aucun package Python tiers ni accès Internet n'est nécessaire. Depuis le répertoire `Dmodel_eLLM_v0` :

```
python run_eLLM_v0.py serve --host 127.0.0.1 --port 8765
```

Ouvrir ensuite `http://127.0.0.1:8765`. Les scripts `start_unix.sh` et `start_windows.bat` facilitent le lancement. L'écoute loopback est le défaut. Ne pas exposer ce serveur non authentifié à un réseau non fiable.

4.2 Réentraînement et tests

```
python scripts/tune_and_train.py \
  --corpus data/corpus_fr.txt \
  --checkpoint checkpoints/model.json \
  --metrics checkpoints/training_metrics.json \
  --seed 2025 --contexts 1,2,3,4,5 \
  --alphas 0.35,1,3,10,30 --max-vocab 5000 \
  --selection-policy max_context_then_validation_nll
python run_eLLM_v0.py evaluate --corpus chemin/vers/test_distinct.txt
python -m unittest discover -s tests -v
node --check web/app.js
python run_eLLM_v0.py footprint "un exemple"
python run_eLLM_v0.py search \
  "Utilisateur : exemple Assistant :" --max-new-tokens 8
python run_eLLM_v0.py imagine \
  "Utilisateur : exemple Assistant :" --count 3 \
  --max-new-tokens 12 --seed 17
python run_eLLM_v0.py diagnose "Utilisateur : exemple Assistant :"
python run_eLLM_v0.py transport-check
```

Le choix d'hyperparamètres est séparé de l'évaluation finale. Ne jamais choisir sur les résultats test. Pour les expériences, consulter les scripts et JSON enregistrés dans `Amodel_eLLM_v0/`.

4.3 Commandes d'interface et API

L'interface offre le dialogue et les actions Explorer, Imaginer, Diagnostics, métriques, thème, nouvelle conversation et effacement mémoire. Les routes principales sont :

— lecture : `GET /api/status`, `GET /api/metrics`, `GET /api/memory` ;

— génération : `POST /api/chat`, `POST /api/search`, `POST /api/imagine` ;

— vérification : `GET/POST /api/diagnostics`, `POST /api/transport-check` ;

— état : `POST /api/reset`, `POST /api/memory/clear`.

L'API accepte jusqu'à 2048 tokens par demande et 4000 caractères par prompt comme garde-fous opérationnels ; le générateur du modèle n'impose pas lui-même de maximum arbitraire. EOS peut terminer avant la longueur souhaitée.

5 Résultats

5.1 Entraînement principal

Mesure	Résultat
Documents corpus	80
Train / validation / test	64 / 8 / 8
Graine du split	2025
Ordre du checkpoint	5
Lissage α	10
Taille vocabulaire	869
Cibles entraînées	2701
Tokens scorés du test	328
NLL/token test	5.3801
Perplexité test	217.0393
Top-1 test	21.04%
Perplexité unigramme même test	449.9162
Top-1 unigramme même test	5.79%

TABLE 2 – Évaluation gelée du checkpoint distribué.

Le delta de perplexité test baseline moins modèle vaut environ 232.877. Ce résultat ne constitue pas un benchmark externe : le test contient huit paragraphes du même petit corpus.

5.2 Répétitions de partition

Graine	Ordre	α	PPL modèle	Top-1 modèle	PPL uni.	Top-1 uni.
2025	5	10.0	217.039	21.04%	449.916	5.79%
7	5	10.0	295.967	14.03%	430.436	5.67%
42	5	10.0	290.436	14.33%	438.909	6.30%

TABLE 3 – Même corpus source, splits documentaires différents ; ce ne sont pas des réplications sur des corpus indépendants.

5.3 Sensibilité au prompt et choix de l'ordre

L'ablation sur les contextes $0, 1, 2, 3, 4, 5, 8, 12$ a montré qu'une meilleure NLL de validation seule ne garantit pas la sensibilité aux différents prompts. Dans le rapport de conditionnement, les contextes 0, 1 et 2 produisent une distribution de prochain token identique pour les huit prompts diagnostiques ; à l'ordre 5, la variation totale moyenne paire-à-paire est d'environ 0.0668 et trois distributions exactes distinctes sont observées. L'ordre 5 est retenu comme compromis d'interaction pour l'artefact livré, non comme optimum universel ni comme preuve de compréhension. Les sorties observées peuvent rester incohérentes ou courtes.

5.4 Invariants des empreintes

Contrôle	Résultat final
États du domaine fini	80
Nombre de classes aux résolutions 0,1,2,3,4,5	1, 53, 79, 80, 80, 80
Violations de raffinement	0
Incohérences seuil–filtration	0
Triplets contrôlés	512000
Violations de la loi du minimum double	0
Factorisation à l'ordre 5 sur le domaine fini	Oui
Factorisation à résolution 0	Non

TABLE 4 – Résultat du vérificateur final après correction du cas limite des séquences identiques/courtes.

Ces nombres dépendent du domaine de 80 préfixes construit par le manifeste. Ils ne certifient pas les propriétés sur l'ensemble des textes possibles. Les signatures tokenisées sont celles consommées réellement par le prédicteur, avec traitement des tokens inconnus.

5.5 Serveur, CLI et interface

Le smoke test du serveur réel a renvoyé HTTP 200 pour la page, status, chat, recherche et imagination ; la réponse chat contient une trace à six niveaux. Recherche a produit deux candidats tracés, imagination deux trajectoires avec graines 170 et 171 ; reset et effacement mémoire ont aussi répondu. Les commandes CLI correspondantes ont renvoyé du JSON valide. Le smoke d'intégration plus large inclut diagnostics, transport-check, métriques et la validation du garde-fou HTTP.

— Suite finale : 38 tests unitaires/HTTP passés, 0 échec.

— Domaine structural canonique : 80 états préfixes, classes par résolution 1/53/79/80/80/80, zéro violation de raffinement et de cohérence seuil–filtration, 512000 triplets et zéro violation du minimum double. Le résultat de régression précédent 1/56/80/80/80/80 provient d'un autre manifeste de préfixes et reste archivé séparément.

— Compilation Python : réussie ; syntaxe JavaScript via Node : réussie.

— Test Chromium : PASS, largeur bureau 1440 px et mobile 390 px, aucun débordement horizontal, aucun message d'erreur navigateur enregistré ; le test DOM injecte des réponses API simulées.

— Une extraction propre d'un candidat du paquet runtime a réussi : aide CLI, empreinte, diagnostic, conversation CLI et suite intégrée 38/38 PASS sans paquet Python tiers. Le hash de ce candidat historique est conservé dans le JSON d'audit ; le hash et le smoke test du ZIP final exact sont fournis séparément dans les vérifications de livraison.

— Réentraînement exact : le SHA-256 du checkpoint reconstruit est identique au checkpoint de livraison : `3e42677b469937ba688022c30d968752c412c09eaabe3d826d2712f6e84590dc`.

5.6 Test de portabilité après extraction propre

Le paquet `Dmodel_eLLM_v0.zip` a été extrait dans un répertoire temporaire neuf à partir de l'archive exacte de hash SHA-256 `77aca7017cf796dd095157946f9526623243bfe87afb9a73245ced86469975c6`. Sans surcharge de `PYTHONPATH`, la suite de tests du paquet extrait retourne `Ran 38 tests ... OK` (38/38), et l'aide CLI présente les neuf commandes sans erreur. Le serveur a ensuite été démarré depuis cette même extraction, et les huit contrôles HTTP ont réussi : disponibilité du modèle, racine HTML, JavaScript statique, chat, recherche, imagination, reset et effacement mémoire. La preuve `Amodel_eLLM_v0/experiments/clean_extraction_portability.json` associe ces résultats au hash exact de l'archive testée. Deux reconstructions successives du paquet runtime, à partir des mêmes sources et de la même politique de manifeste reproductible, ont rendu le même SHA-256 ; voir `Amodel_eLLM_v0/experiments/package_reproducibility.json`. Cette vérification établit la portabilité et la reproductibilité du paquet dans l'environnement d'essai observé ; elle ne constitue pas une preuve exhaustive sur tous les systèmes d'exploitation et navigateurs.

6 Interprétations des résultats

Les résultats indiquent que, sur le corpus et les partitions mesurés, le modèle d'ordre 5 améliore la perplexité et la précision top-1 par rapport au baseline unigramme. Trois graines sur le même corpus conservent le sens de cette différence, mais ne remplacent pas des tests sur d'autres corpus. Les partitions induites par les signatures suffixes se raffinent exactement dans le domaine fini. La loi du minimum double est satisfaite pour tous les triplets testés ; l'expérience vérifie l'implémentation de cette construction, pas une théorie universelle de la sémantique. La sensibilité du prompt est limitée : seulement trois distributions distinctes sur les huit questions de diagnostic à l'ordre 5 dans l'expérience conservée. Un générateur statistique peut produire des continuations plausibles localement sans répondre à l'intention de la question. Aucune mesure d'évaluation humaine ni factualité indépendante n'est disponible. La réussite de l'API et de l'interface établit leur intégration fonctionnelle dans l'environnement testé, pas une absence absolue de bugs ou une certification universelle « 100

7 Conséquences théoriques, conceptuelles, opérationnelles

7.1 Théoriques

Le lien entre famille de sondes, signature, partition et seuil est opérationnel et vérifiable. L'empreinte discrète encode une chaîne de suffixes emboîtés ; la distribution prédictive continue reste une observable séparée. La factorisation est une propriété de constance sur les fibres d'un ensemble fini fourni. Les résultats ne démontrent pas que toute connaissance ou généralisation d'un modèle de langage se ramène à ces signatures.

7.2 Conceptuelles

Le code ne traite aucune unité interne comme un concept sémantique certifié. La polysémanticité, la mémoire distribuée, les invariants de sens et l'intelligence émergente restent des hypothèses empiriques ou des interprétations, non des faits validés par ces expériences. Le composant `diagnostics` renvoie des proxies distincts (observation, capacité, résolution, recherche, transfert, contraintes), pas une mesure psychométrique ou une ontologie exhaustive.

7.3 Opérationnelles

Le modèle s'exécute localement sans paquet Python tiers et sans service distant. Le checkpoint est versionné, reproductible, mesurable et livré avec source et tests. Les journaux conservent aussi

les incidents corrigés : erreur de test utilisant un token connu, délai de la grille initiale, mauvais répertoire d'exécution, harnais Chromium, assertion sur contenu replié, commande d'agrégation mal formée, copie d'un fichier vers lui-même et absence de parent lors du snapshot, ainsi que le cas limite seuil–filtration. Ces incidents ont été corrigés et les tests de non-régression ont passé ; ils restent dans le journal au lieu d'être effacés. Le test Chromium utilise des réponses simulées, tandis que l'API réelle est validée séparément. La mémoire et les logs sont locaux ; le serveur n'a pas d'authentification et doit rester en loopback dans l'usage livré.

8 Conclusions

Un pipeline exécutable de bout en bout est livré : entraînement par comptages, checkpoint, prédiction/génération, empreintes exactes à plusieurs résolutions, seuils/quotients, outils de recherche et imagination, diagnostics, factorisation/commutation, API HTTP, UI, tests, rapports et preuves. Les résultats quantitatifs sont positifs par rapport au baseline unigramme pour le petit test conservé et les invariants structurels annoncés passent sur leur domaine fini. Les faiblesses mesurées, en particulier la faible sensibilité conditionnelle aux prompts et le corpus étroit, restent significatives. Aucun résultat ne doit être extrapolé sans nouvel essai préenregistré et indépendant.

9 Statut du modèle

— **Implémenté** : prédicteur n-gramme interpolé, tokenisation, apprentissage, métriques, checkpoint, signatures/partitions/seuils/fermeture/maillage finis, mémoire locale facultative, recherche, imagination, diagnostics, transport-check, API, interface locale et sauvegarde.

— **Testé dans l'environnement** : 38 tests, compilation, Chromium DOM avec API simulée, smoke HTTP/CLI, réentraînement hash-identique, invariants finis et nouvelle vérification de portabilité depuis le ZIP extrait.

— **Partiellement validé** : sensibilité au prompt et utilité des continuations, sur quelques essais prédéfinis seulement.

— **Non revendiqué** : performance générale, compréhension humaine, factualité, raisonnement long, sûreté exhaustive, généralisation sémantique universelle, compétition avec un Transformer ou fiabilité sur des corpus non vus.

— **Conclusion de statut** : prototype expérimental autonome et reproductible dans les conditions observées, pas un LLM neuronal de grande échelle. Ce statut décrit la version actuelle sans prédire le potentiel de versions futures.

Annexe A – Artefacts de preuve

Le paquet audit contient le JSON des métriques, le résumé multi-graine, l'ablation de contexte, la sensibilité au prompt, le vérificateur structurel, le smoke serveur/CLI, le test navigateur, la reproductibilité du checkpoint, les scripts et les logs d'échecs corrigés. La carte complète concept–fonction–ligne–test–résultat est dans `prouves_eLLM_v0.json`. Les sommes SHA-256 figurent dans `manifestes_SHA256_eLLM_v0.json`.